Sentiment Classification of Airline Tweets Using Deep Learning

Machine Learning: Developing a Solution Suited to the Problem

WALID M. W. KOBAHALABI | AI Internship, Software Persona | October 2026

Code: github.com/WalidMWH/airline-sentiment-dl

1. Problem Definition

Airlines receive a large number of tweets every day, and many of them are complaints that need a quick reply. Reading all of them by hand is slow, so in this project I built a deep learning model that reads the text of a tweet and classifies it as negative, neutral or positive. The input is the tweet text and the output is one of these three classes. A model like this could help a customer service team sort incoming tweets and answer unhappy customers first. For this reason I paid special attention to how many negative tweets the model actually catches, and not only to the overall accuracy.

2. Dataset

I used the Twitter US Airline Sentiment dataset from Kaggle. It contains 14,640 tweets sent to six US airlines, and each tweet was labeled by human annotators. I used the *text* column as the input and *airline_sentiment* as the target. Other columns such as the airline name and the negative reason were only used for analysis and were never given to the model. There were no missing values, but I found 213 duplicate tweets.

Dataset link: https://www.kaggle.com/datasets/crowdflower/twitter-airline-sentiment

Table 1. Class distribution of the dataset

Class	Number of tweets	Share
Negative	9,178	62.7%
Neutral	3,099	21.2%
Positive	2,363	16.1%

The data is clearly imbalanced. There are almost four times more negative tweets than positive ones, so a model that always predicts "negative" would already reach 62.7% accuracy. Because of this I used macro F1 as my main metric, since it gives the same weight to each class.

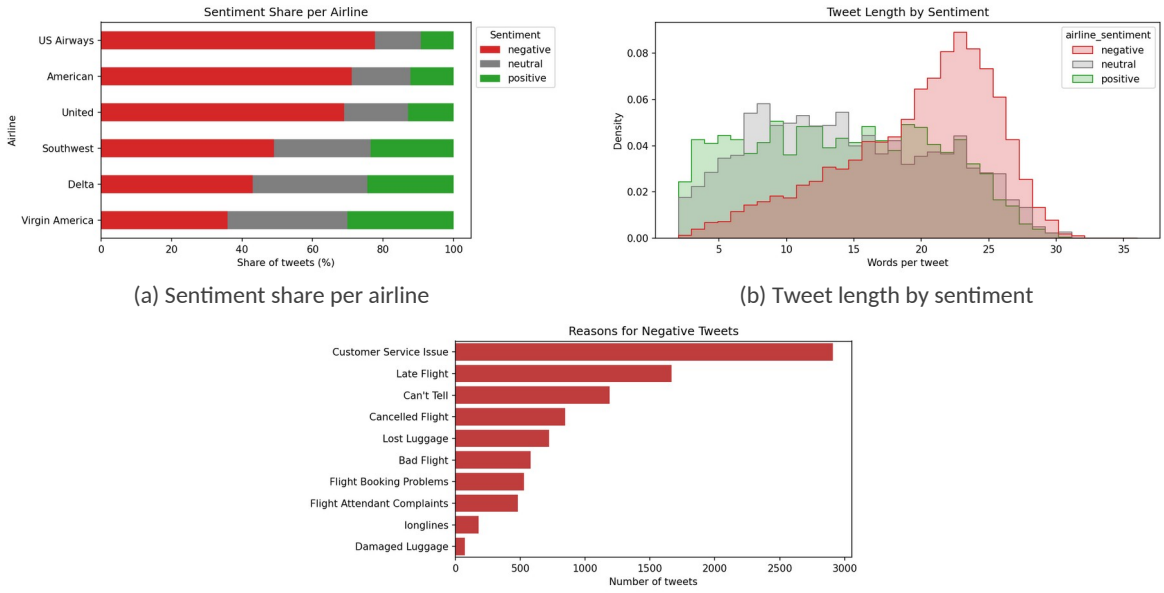
3. Data Analysis and Preprocessing

3.1 Data Exploration

Before building any model I explored the data with three charts (Figure 1). The first thing I noticed was that the share of negative tweets depends a lot on the airline (Figure 1a): it is about 78% for US Airways but only around 36% for Virgin America. Since every tweet starts with the airline's @ handle, a model could learn that "@usairways" means negative without reading the tweet, so I removed all @mentions.

The second chart (Figure 1b) shows that negative tweets are longer than the others, with 19.7 words on average compared to 14.4 for neutral and 14.0 for positive tweets. People who complain usually explain what went wrong, while happy customers just say thanks. Since 95% of the tweets have 27 words or fewer, I set the maximum sequence length to 30 tokens.

Finally, I looked at the reasons given for negative tweets (Figure 1c). Customer service issues (about 2,900) and late flights (about 1,650) were the most common ones. Around 1,190 negative tweets have the reason "Can't Tell", which means the annotators agreed the tweet was negative but could not say why. I took this as a first sign that some labels might be noisy.



(a) Sentiment share per airline

(b) Tweet length by sentiment

(c) Reasons given for negative tweets

Figure 1. Exploratory analysis of the dataset

3.2 Preprocessing

I applied the following steps:

- Removed the 213 duplicate tweets, so the same tweet could not appear in both the training and the test data. This left 14,427 tweets.
- Converted the text to lowercase and removed URLs, @mentions and the # symbol (the hashtag word itself was kept). I kept "!" and "?" as separate tokens because they show emotion, and I did not remove stopwords, because words like "not" and "no" can change the meaning of a sentence completely.
- Split the data into training (70%), validation (15%) and test (15%) sets using stratification, so all three sets have the same class percentages. This gave 10,098, 2,164 and 2,165 tweets.
- Built a vocabulary of the 10,000 most frequent words using only the training set, and converted every tweet into a sequence of 30 word IDs with zero padding.
- Calculated balanced class weights (0.53 for negative, 1.57 for neutral and 2.10 for positive), so that mistakes on the smaller classes cost more during training.

One weakness of my cleaning is that it removes emojis, which often carry sentiment. For example, "you guys are so clever" was followed by a smiling emoji in the original tweet, but the emoji was lost after cleaning.

4. Model Selection and Design

I built two models to answer one question: does it help to read the words in order, or is it enough to know which words appear in the tweet? Both models start with the same embedding layer and end with the same classifier, so the only difference is the middle layer (Table 2).

The embedding layer learns a 64-dimensional vector for every word, and I used masking so the padding zeros are ignored. The baseline simply averages these vectors, so it treats the tweet as a "bag of words" and cannot see the difference between "good" and "not good". The BiLSTM reads the tweet from left to right and from right to left, so every word is seen together with its context. I expected this to help with negations.

For both models I used ReLU in the hidden layer and softmax in the output layer to get a probability for each class. The loss function is sparse categorical cross-entropy, since the labels are the integers 0, 1 and 2, and the optimizer is Adam with a learning rate of 0.001. Because about 90% of the parameters are in the embedding layer, I added dropout to reduce overfitting.

Table 2. Architecture of the two models

Layer	Baseline (bag of words)	BiLSTM
Input	30 word IDs	30 word IDs
Embedding	10,000 words, 64 dimensions	10,000 words, 64 dimensions
Middle layer	Global average pooling	Bidirectional LSTM (64 units each way)
Hidden layer	Dense, 32 units, ReLU	Dense, 32 units, ReLU
Dropout (two layers)	0.3	0.4
Output	Dense, 3 units, softmax	Dense, 3 units, softmax
Parameters	642,179	710,275

5. Training Process

I trained both models in Google Colab on a T4 GPU, using TensorFlow 2.21 and a fixed random seed of 42. I used a batch size of 64, up to 30 epochs, the class weights from Section 3.2, and early stopping on the validation loss (patience of 3 epochs, restoring the best weights).

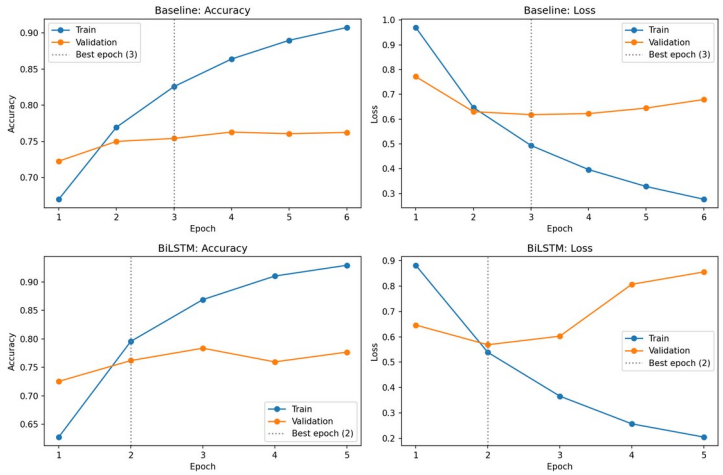


Figure 2. Training and validation curves of both models

As Figure 2 shows, both models started to overfit very quickly. Training accuracy kept rising above 90%, but validation accuracy stayed around 76% and the validation loss started to increase after the second or third epoch. Early stopping kept epoch 3 for the baseline (validation loss 0.618) and epoch 2 for the BiLSTM (validation loss 0.569).

Interestingly, at epoch 3 the BiLSTM's validation accuracy rose to 0.784 while its validation loss also rose, meaning it got a few more tweets right but became overconfident on the wrong ones. This is why I used the validation loss for early stopping.

6. Evaluation Results

I evaluated both models once on the test set (2,165 tweets) and compared them with a majority-class baseline that always predicts "negative" (Table 3).

Table 3. Results on the test set

Model	Accuracy	Macro precision	Macro recall	Macro F1	F1 negative	F1 neutral	F1 positive
Majority class	0.630	0.210	0.333	0.258	0.773	0.000	0.000
Baseline	0.752	0.689	0.737	0.705	0.837	0.599	0.680
BiLSTM	0.750	0.697	0.728	0.706	0.832	0.594	0.694

Both models are clearly better than the majority-class baseline. Macro F1 went from 0.258 to about 0.705, while accuracy only improved from 63% to 75%, which shows how misleading accuracy is on imbalanced data.

The result I did not expect was that the BiLSTM was not better than the simple baseline. The difference in macro F1 is only 0.001, which is too small to mean anything from a single training run. I think the reason is that tweets are short and their sentiment usually comes from a few strong words like "thanks", "delayed" or "worst", which the baseline can already pick up without knowing the word order. Since the baseline is just as accurate and faster, I would choose it for this task.

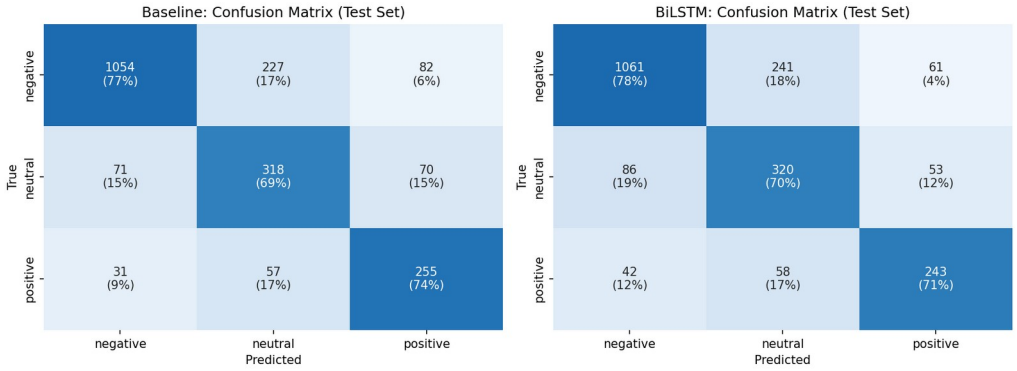


Figure 3. Confusion matrices on the test set (count and percentage of each true class)

The confusion matrices in Figure 3 show where the models make mistakes. Neutral is the weakest class, with an F1 of about 0.60 and a precision of only around 0.52. The most common error is a negative tweet predicted as neutral (241 tweets for the BiLSTM). I think two things cause this. First, the class weight for neutral is three times higher than for negative, so the model leans towards neutral when it is unsure. Second, many complaints are written politely, for example as a question asking for help.

For negative tweets the precision is high (about 0.90) but the recall is around 0.78, so the model misses roughly one complaint out of four. For an airline this is the most important number to improve. On the other hand, negative and positive tweets are rarely confused with each other, which shows that the models understand the general direction of the sentiment and mainly struggle with the borderline neutral cases.

7. Error Analysis

The BiLSTM got 541 of the 2,165 test tweets wrong (25%), and 438 of these errors (81%) involve the neutral class. To understand the mistakes better, I read the 15 tweets where the model was wrong but most confident (0.95 to 0.99 probability) and grouped them by hand (Table 4).

Table 4. The 15 most confident mistakes, grouped by cause

Cause	Count	Example (true label, predicted label)
Wrong or debatable label	7	"sitting on a plane in Philadelphia for over 20 minutes waiting just to get off the plane. Great service!" (positive, negative)
Mixed feelings	2	"That made me so mad, but then I called the bag number, and that person was helpful..." (positive, negative)
Complaint written as a question	2	"How do we add guitars and instruments to the list?" (negative, neutral)
Unknown words or hashtags	2	"Mold on my flight?!? #filthyplane" (negative, neutral)
Dataset artifact	1	"I need to leave from Hawaii Late Flight evening" (neutral, negative)
Unclear even for a human	1	"...a percentage on every plastic bottle... turn up the heat" (neutral, negative)

The biggest group surprised me. In 7 of the 15 cases I would actually agree with the model rather than with the label. The "Great service!" tweet is clearly sarcastic, and the model correctly called it negative

while the annotator labeled it as positive. This means that part of the error comes from noisy labels, and the real performance of the model is probably a bit higher than the numbers show.

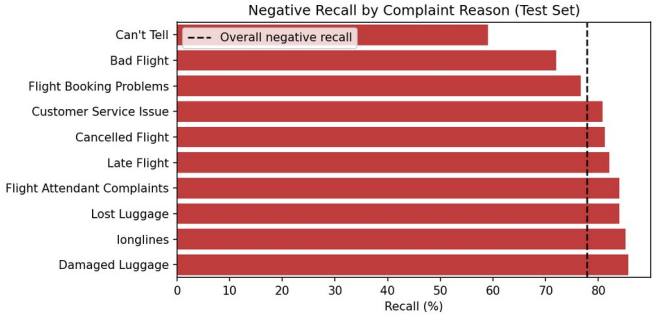


Figure 4. Recall on negative tweets by complaint reason (BiLSTM, test set)

I also checked which kinds of complaints the model misses (Figure 4). Complaints with clear keywords, such as lost or damaged luggage and late flights, are recognized 82% to 86% of the time. Vaguer complaints are missed more often: 72% for "Bad Flight" and only 59% for "Can't Tell". If the annotators could not say why a tweet was negative, it makes sense that the model struggles too.

While reading the errors I also found a problem in the dataset itself. The tweet "leave from Hawaii Late Flight evening" did not make sense, so I searched the data and found that the word "late" had been replaced with "Late Flight" and "cancel" with "Cancelled Flight", probably by an automatic find-and-replace using the complaint category names. This affects 447 and 999 tweets and creates strange words such as "Late Flightr" (from "later") and "Cancelled Flighted". As a result, neutral sentences like "late evening" get a false negative signal.

8. Conclusion and Improvements

In this project I built and compared two deep learning models for classifying airline tweets. Both reached a macro F1 of about 0.71 and an accuracy of 75%, compared to a macro F1 of 0.26 for a majority-class guess. The BiLSTM did not perform better than the simpler bag-of-words model, so for this data the simpler model is the better choice. Many of the remaining errors come from noisy labels and the "Late Flight" artifact rather than from the models.

If I continued this project, I would try the following improvements:

- Improve the preprocessing: undo the "Late Flight" and "Cancelled Flight" replacements, convert emojis to words instead of deleting them, and split hashtags such as "#filthyplane" into separate words.
- Use pretrained word embeddings such as GloVe Twitter to reduce overfitting.
- Lower the decision threshold for the negative class to catch more complaints.
- Try a pretrained transformer such as BERT, which handles context and sarcasm better.
- Use the annotator confidence score in the dataset to remove or down-weight uncertain labels.

The full notebook, figures and code are available on GitHub:
https://github.com/WalidMWH/airline-sentiment-dl